

Piotr's TechBlog

Java, Spring, Kotlin, microservices, Kubernetes, containers

Tool Calling with Spring AI

AI, Spring Boot, Uncategorized

Tool Calling with Spring AI

By piotr.minkowski • March 13, 2025 • 2



Search

Search

Go

Follow Blog via Email

Enter your email address to follow this blog and receive notifications of new posts by email.

Enter your email address

This article will show you how to use Spring AI support with the most popular AI models for the tool calling feature. Tool calling (or function calling), is a common pattern in AI applications that enables a model to interact with APIs or tools, extending its capabilities. The most popular AI models are trained to know when to call a function. Spring AI formerly supported it through the Function Calling API, which has been deprecated and marked for removal in the next release. My previous [article](#) described that feature based on interactions with an internal database and an external market stock API. Today, we will consider the same use case. This time, however, we will replace the deprecated Function Calling API with a new **Tool calling** feature.

This is the sixth part of my series of articles about Spring Boot and AI. It is worth reading the following posts before proceeding with the current one. Please pay special attention to the second article. I will refer to it often in this article.

1. <https://piotrminkowski.com/2025/01/28/getting-started-with-spring-ai-and-chat-model>: The first tutorial introduces the Spring AI project and its support for building applications based on chat models like OpenAI or Mistral AI.
2. <https://piotrminkowski.com/2025/01/30/getting-started-with-spring-ai-function-calling>: The second tutorial shows Spring AI support for Java function calling with the OpenAI chat model.
3. <https://piotrminkowski.com/2025/02/24/using-rag-and-vector-store-with-spring-ai>: The third tutorial shows Spring AI support for RAG (Retrieval Augmented Generation) and vector store.
4. <https://piotrminkowski.com/2025/03/04/spring-ai-with-multimodality-and-images>: The fourth tutorial shows Spring AI support for a multimodality feature and image generation
5. <https://piotrminkowski.com/2025/03/10/using-ollama-with-spring-ai>: The fifth tutorial shows Spring AI supports for interactions with AI models run with Ollama

Source Code

Feel free to use my source code if you'd like to try it out yourself. To do that, you must clone my sample GitHub [repository](#). Then you should only follow my instructions.

Follow

Categories

- AI (18)
- Cloud (42)
- Containers (67)
- Continuous Integration (37)
- Data Grids (7)
- Kotlin (9)
- Kubernetes (115)
- Message Brokers (27)
- Micronaut (12)
- Microservices (64)
- Other (28)
- Performance (28)
- Quarkus (18)
- Security (26)
- Spring Boot (113)
- Spring Cloud (51)
- Uncategorized (1)

Motivation for Tool Calling in Spring AI

The tool calling feature helps us solve a common AI model challenge related to internal or live data sources. If we want to augment a model with such data our applications must allow it to interact with a set of APIs or tools. In our case, the internal database (H2) contains information about the structure of our stock wallet. The sample Spring Boot application asks an AI model about the total value of the wallet based on daily stock prices or the highest value for the last few days. The model must retrieve the structure of our stock wallet and the latest stock prices. We will do the same exercise as for a function calling feature. It will be enhanced with additional scenarios I'll describe later.

Use the Calling Tools Feature in Spring AI

Create WalletTools

Let's begin with the `WalletTools` implementation, which is responsible for interaction with a database. We can compare it to the previous implementation based on Spring functions available in the `pl.piomin.services.functions.stock.WalletService` class. It defines a single method annotated with `@Tool`. The important element is the right description that must inform the model what that method does. The method returns the number of shares for each company in our portfolio retrieved from the database through the Spring Data `@Repository`.

```
public class WalletTools {  
  
    private WalletRepository walletRepository;  
  
    public WalletTools(WalletRepository walletRepository) {  
        this.walletRepository = walletRepository;  
    }  
  
    @Tool(description = "Number of shares for each co  
    public List<Share> getNumberOfShares() {  
        return (List<Share>) walletRepository.findAll
```

Tags

[Apache Kafka](#)[argocd](#)[Consul](#)[Continuous Delivery](#)[Continuous Integration](#)[Docker](#)[Eureka](#)[gitops](#)[Helm](#)[istio](#)[Java](#)[Jenkins](#)[Jib](#)[JPA](#)[kafka](#)[kind](#)[Knative](#)[Kotlin](#)[Kubernetes](#)[Maven](#)[Micronaut](#)[microservices](#)[Minikube](#)[OAuth2](#)[open-ai](#)[openshift](#)[Performance](#)[postgresql](#)[Prometheus](#)[quarkus](#)[Reactor](#)[REST](#)[security](#)[Scaffold](#)[Spring](#)[spring-ai](#)[Spring Boot](#)[Spring Boot Actuator](#)[Spring Cloud](#)[Spring Cloud Gateway](#)[Spring Cloud Kubernetes](#)[Spring Data](#)[testcontainers](#)[testing](#)[Vault](#)

```
}  
}
```

We can register the `WalletTools` class as a Spring `@Bean` in the application main class.

```
@Bean  
public WalletTools walletTools(WalletRepository walletRepository) {  
    return new WalletTools(walletRepository);  
}
```

The Spring Boot application launches an embedded, in-memory database and inserts test data into the stock table. Our wallet contains the most popular companies on the U.S. stock market, including Amazon, Meta, and Microsoft.

```
insert into share(id, company, quantity) values (1, 'Amazon', 100);  
insert into share(id, company, quantity) values (2, 'Meta', 100);  
insert into share(id, company, quantity) values (3, 'Microsoft', 100);  
insert into share(id, company, quantity) values (4, 'Apple', 100);  
insert into share(id, company, quantity) values (5, 'Google', 100);
```

Create StockTools

The `StockTools` class is responsible for interaction with TwelveData stock API. It defines two methods. The `getLatestStockPrices` method returns only the latest close price for a specified company. It is a tool calling version of the method provided within the `pl.piomin.services.functions.stock.StockService` function. The second method is more complicated. It must return a historical daily close prices for a defined number of days. Each price must be correlated with a quotation date.

```
public class StockTools {  
  
    private static final Logger LOG = LoggerFactory.getLogger(StockTools.class);  
  
    private RestTemplate restTemplate;  
    @Value("${STOCK_API_KEY:none}")  
    String apiKey;  
  
    public StockTools(RestTemplate restTemplate) {
```

```

        this.restTemplate = restTemplate;
    }

    @Tool(description = "Latest stock prices")
    public StockResponse getLatestStockPrices(@ToolParam StockData data = restTemplate.getForObject("https://api.iexcloud.com/stock/price/v1/{company}?apiKey={apiKey}", StockData.class, company, apiKey);

    DailyStockData latestData = data.getValues().
    LOG.info("Get stock prices: {} → {}", company, latestData);
    return new StockResponse(Float.parseFloat(latestData.getPrice()));
}

```

```

    @Tool(description = "Historical daily stock price")
    public List<DailyShareQuote> getHistoricalStockPrices(@ToolParam StockData data = restTemplate.getForObject("https://api.iexcloud.com/stock/price/v1/{company}?apiKey={apiKey}", StockData.class, company, days, apiKey);

    return data.getValues().stream()
        .map(d → new DailyShareQuote(company, d.getDate(), d.getPrice()))
        .toList();
}
}

```

Here's the `DailyShareQuote` Java record returned in the response list.

```

public record DailyShareQuote(String company, float price, Date date) {}

```

Then, let's register the `StockUtils` class as a Spring `@Bean`.

```

@Bean
public StockTools stockTools() {
    return new StockTools(restTemplate());
}

```

Spring AI Tool Calling Flow

Here's a fragment of the `WalletController` code, which is responsible for defining interactions with LLM and HTTP endpoints implementation. It injects both `StockTools` and `WalletTools` beans.

```

@RestController
@RequestMapping("/wallet")
public class WalletController {

    private final ChatClient chatClient;
    private final StockTools stockTools;
    private final WalletTools walletTools;

    public WalletController(ChatClient.Builder chatCl
                          StockTools stockTools,
                          WalletTools walletTools)
    {
        this.chatClient = chatClientBuilder
            .defaultAdvisors(new SimpleLoggerAdvi
            .build();
        this.stockTools = stockTools;
        this.walletTools = walletTools;
    }

    // HTTP endpoints implementation
}

```

The `GET /wallet/with-tools` endpoint calculates the value of our stock wallet in dollars. It uses the latest daily stock prices for each company's shares from the wallet. There are a few ways to register tools for a chat model call. We use the tools method provided by the `ChatClient` interface. It allows us to pass the tool object references directly to the chat client. In this case, we are registering the `StockTools` bean which contains two `@Tool` methods. The AI model must choose the right method to call in `StockTools` based on the description and input argument. It should call the `getLatestStockPrices` method.

```

@GetMapping("/with-tools")
String calculateWalletValueWithTools() {
    PromptTemplate pt = new PromptTemplate("""
    What's the current value in dollars of my wallet b
    """);

    return this.chatClient.prompt(pt.create())
        .tools(stockTools, walletTools)
        .call()
        .content();
}

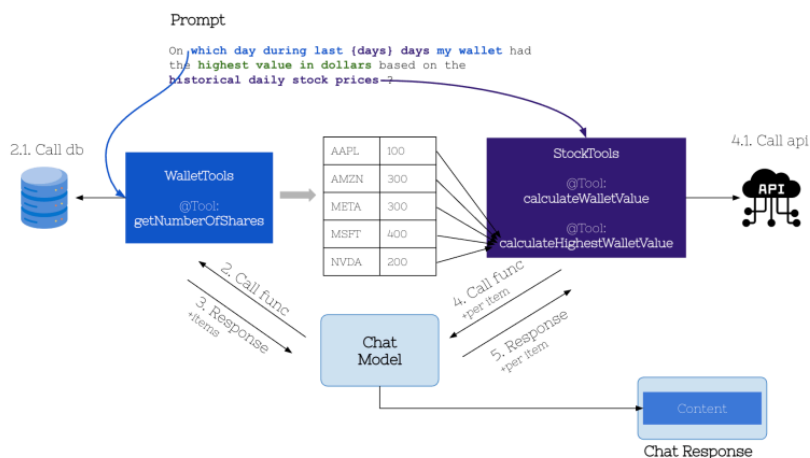
```

The `GET /wallet/highest-day/{days}` endpoint calculates the value of our stock wallet in dollars for each day in the specified period determined by the `days` variable. Then it must return the day with the highest stock wallet value. Same as before we use the `tools` method from `ChatClient` to register our tool calling methods. It should call the `getHistoricalStockPrices` method.

```
@GetMapping("/highest-day/{days}")
String calculateHighestWalletValue(@PathVariable int
    PromptTemplate pt = new PromptTemplate("""
    On which day during last {days} days my wallet had
    """);

    return this.chatClient.prompt(pt.create(Map.of("da
        .tools(stockTools, walletTools)
        .call()
        .content();
}
```

The following diagram illustrates a flow for the second use case that returns the day with the highest stock wallet value. First, it must connect with the database and retrieve the stock wallet structure containing a number of each company shares. Then, it must call the stock API for every company found in the wallet. So, finally, the method `calculateHighestWalletValue` should be called five times with different values of the `company @ToolParam` and a value of the days determined by the HTTP endpoint path variable. Once all the data is collected AI model calculates the highest wallet value and returns it together with the quotation date.



Run Application and Verify Tool Calling

Before starting the application we must set environment variables with the AI model and stock API tokens.

```
export OPEN_AI_TOKEN=<YOUR_OPEN_AI_TOKEN>
export STOCK_API_KEY=<YOUR_STOCK_API_KEY>
```

Then run the following Maven command:

```
mvn spring-boot:run
```

Once the application is started, we can call the first endpoint. The `GET /wallet/with-tools` calculates the total least value of the stock wallet structure stored in the database.

```
curl http://localhost:8080/wallet/with-tools
```

Here's the fragment of logs generated by the Spring AI `@Tool` methods. The model behaves as expected. First, it calls the `getNumberOfShares` tool to retrieve a wallet structure. Then it calls the `getLatestStockPrices` tool per share to obtain its current price.

```
2025-03-13T15:14:27.750+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.s.tool.DefaultToolCallingManager : Executing tool call: getNumberOfShares
2025-03-13T15:14:27.750+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Starting execution of tool: getNumberOfShares
2025-03-13T15:14:27.795+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Successful execution of tool: getNumberOfShares
2025-03-13T15:14:27.795+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
2025-03-13T15:14:29.039+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.s.tool.DefaultToolCallingManager : Executing tool call: getLatestStockPrices
2025-03-13T15:14:29.039+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Starting execution of tool: getLatestStockPrices
2025-03-13T15:14:29.235+01:00 INFO 10410 --- [nio-8080-exec-1] pl.piomin.services.tools.StockTools : Get stock prices: AAPL -> 214.95
2025-03-13T15:14:29.235+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Successful execution of tool: getLatestStockPrices
2025-03-13T15:14:29.237+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
2025-03-13T15:14:29.237+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.s.tool.DefaultToolCallingManager : Executing tool call: getLatestStockPrices
2025-03-13T15:14:29.237+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Starting execution of tool: getLatestStockPrices
2025-03-13T15:14:29.299+01:00 INFO 10410 --- [nio-8080-exec-1] pl.piomin.services.tools.StockTools : Get stock prices: AMZN -> 195.855
2025-03-13T15:14:29.299+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Successful execution of tool: getLatestStockPrices
2025-03-13T15:14:29.299+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
2025-03-13T15:14:29.300+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.s.tool.DefaultToolCallingManager : Executing tool call: getLatestStockPrices
2025-03-13T15:14:29.300+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Starting execution of tool: getLatestStockPrices
2025-03-13T15:14:29.374+01:00 INFO 10410 --- [nio-8080-exec-1] pl.piomin.services.tools.StockTools : Get stock prices: META -> 598.73
2025-03-13T15:14:29.374+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.ai.tool.method.MethodToolCallback : Successful execution of tool: getLatestStockPrices
2025-03-13T15:14:29.374+01:00 DEBUG 10410 --- [nio-8080-exec-1] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
```

Here's a final response with a wallet value with a detailed explanation.

```
~ (13.828s)
curl http://localhost:8080/wallet/with-tools
Here's the current value in dollars of your wallet based on the latest stock prices:

- **AAPL**: 100 shares x $214.64 = $21,464.00
- **AMZN**: 300 shares x $194.80 = $58,440.00
- **META**: 300 shares x $598.71 = $179,613.00
- **MSFT**: 400 shares x $382.84 = $153,136.00
- **NVDA**: 200 shares x $115.44 = $23,088.00

Now, let's sum these amounts:

\[
21,464 + 58,440 + 179,613 + 153,136 + 23,088 = 435,741
\]

Therefore, the total value of your wallet is **$435,741.00**.
```


Then we can call the `GET /wallet/highest-day/{days}` endpoint to return the day with the highest wallet value. Let's calculate it for the last 20 days.

```
curl http://localhost:8080/wallet/highest-day/20
```

The response is very detailed. Here's the final part of the content returned by the OpenAI chat model. It returns 26.02.2025 as the day with the highest wallet value. Frankly, sometimes it returns different answers...

```
10. **2025-02-20** \ ( (241.04 \times 100) + (212.20 \times 100) + (668.20 \times 100) + (396.99 \times 100) + (124.92 \times 100) = 24184 +
63684 + 200460 + 158796 + 24984 = 420108 \ )
11. **2025-02-27** \ ( (237.30 \times 100) + (208.74 \times 100) + (658.24 \times 100) + (392.53 \times 100) + (120.15 \times 100) = 23730 +
62622 + 197472 + 157812 + 24036 = 423366 \ )
12. **2025-02-26** \ ( (248.36 \times 100) + (214.35 \times 100) + (673.70 \times 100) + (399.73 \times 100) + (131.28 \times 100) = 24036 +
64385 + 202110 + 159802 + 26256 = 440592 \ )
13. **2025-02-25** \ ( (247.84 \times 100) + (212.80 \times 100) + (657.50 \times 100) + (397.90 \times 100) + (126.63 \times 100) = 24784 +
63840 + 197250 + 159160 + 25326 = 420280 \ )
14. **2025-02-24** \ ( (247.10 \times 100) + (212.71 \times 100) + (668.13 \times 100) + (404.00 \times 100) + (130.28 \times 100) = 24710 +
63813 + 200439 + 161600 + 26055 = 409116 \ )
15. **2025-02-21** \ ( (245.55 \times 100) + (216.58 \times 100) + (683.55 \times 100) + (408.21 \times 100) + (134.43 \times 100) = 24555 +
64974 + 205805 + 163284 + 26886 = 409185 \ )
16. **2025-02-20** \ ( (245.82 \times 100) + (222.80 \times 100) + (694.84 \times 100) + (416.13 \times 100) + (140.11 \times 100) = 24582 +
66864 + 208452 + 166452 + 28022 = 408019 \ )
17. **2025-02-19** \ ( (244.87 \times 100) + (226.63 \times 100) + (703.77 \times 100) + (414.77 \times 100) + (139.23 \times 100) = 24487 +
67989 + 211131 + 165900 + 27846 = 408361 \ )
18. **2025-02-18** \ ( (244.47 \times 100) + (226.65 \times 100) + (716.37 \times 100) + (409.64 \times 100) + (139.40 \times 100) = 24447 +
67995 + 214911 + 163850 + 27888 = 409189 \ )
19. **2025-02-14** \ ( (244.60 \times 100) + (228.60 \times 100) + (736.67 \times 100) + (408.43 \times 100) + (138.85 \times 100) = 24460 +
68684 + 221801 + 163272 + 27778 = 409297 \ )
20. **2025-02-13** \ ( (241.53 \times 100) + (230.37 \times 100) + (728.56 \times 100) + (410.54 \times 100) + (135.29 \times 100) = 24153 +
69111 + 218508 + 164216 + 27858 = 410406 \ )

From these calculations, the highest value in your wallet during the last 20 days was on **2025-02-26** with a total value of **$440,599**.
```

However, the AI flow works fine. First, it calls the `getNumberOfShares` tool to retrieve a wallet structure. Then it calls the `getHistoricalStockPrices` tool per share to obtain its prices for the last 20 days.

```
2025-03-13T15:21:23.644+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.w.tool.DefaultToolCallingManager : Executing tool call: getNumberOfShares
2025-03-13T15:21:23.647+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Starting execution of tool: getNumberOfShares
2025-03-13T15:21:23.651+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Successful execution of tool: getNumberOfShares
2025-03-13T15:21:23.652+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
2025-03-13T15:21:23.222+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.w.tool.DefaultToolCallingManager : Executing tool call: getHistoricalStockPrices
2025-03-13T15:21:23.222+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Starting execution of tool: getHistoricalStockPrices
2025-03-13T15:21:23.223+01:00 INFO 10430 --- [nio-8080-exec-4] pl.piomin.services.tools.StockTools : Get historical stock prices: AAPL for 20 days
2025-03-13T15:21:23.298+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Successful execution of tool: getHistoricalStockPrices
2025-03-13T15:21:23.298+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
2025-03-13T15:21:23.299+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.w.tool.DefaultToolCallingManager : Executing tool call: getHistoricalStockPrices
2025-03-13T15:21:23.299+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Starting execution of tool: getHistoricalStockPrices
2025-03-13T15:21:23.299+01:00 INFO 10430 --- [nio-8080-exec-4] pl.piomin.services.tools.StockTools : Get historical stock prices: JPM for 20 days
2025-03-13T15:21:23.373+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Successful execution of tool: getHistoricalStockPrices
2025-03-13T15:21:23.373+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.t.e.DefaultToolCallResultConverter : Converting tool result to JSON.
2025-03-13T15:21:23.376+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.w.tool.DefaultToolCallingManager : Executing tool call: getHistoricalStockPrices
2025-03-13T15:21:23.376+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Starting execution of tool: getHistoricalStockPrices
2025-03-13T15:21:23.376+01:00 INFO 10430 --- [nio-8080-exec-4] pl.piomin.services.tools.StockTools : Get historical stock prices: META for 20 days
2025-03-13T15:21:23.450+01:00 DEBUG 10430 --- [nio-8080-exec-4] o.s.a.i.tool.method.MethodToolCallback : Successful execution of tool: getHistoricalStockPrices
```

We can switch to another AI model to compare their responses. You can connect my sample Spring Boot application e.g. with Mistral AI by activating the `mistral-ai` Maven profile.

```
mvn spring-boot:run -Pmistral-ai
```

Before running the app we must export the Mistral API token.

```
export MISTRAL_AI_TOKEN=<YOUR_MISTRAL_AI_TOKEN>
```

To get the best results I changed the Mistral model to `mistral-large-latest`.

```
spring.ai.mistralai.chat.options.model = mistral-large
```

The response from Mistral AI was pretty quick and short:

```
~ ( 5.125s )
curl http://localhost:8080/wallet/with-tools
The current value of your wallet is 567 953 dollars.%
```

Final Thoughts

In this article, we analyzed the Spring AI support for tool calling support, which replaces Function Calling API. Tool calling is a powerful feature that enhances how AI models interact with external tools, APIs, and structured data. It makes AI more interactive and practical for real-world applications. Spring AI provides a flexible way to register and invoke such tools. However, it still requires attention from developers, who need to define clear function schemas and handle edge cases.

[LLM](#)[open-ai](#)[Spring Boot](#)[spring-ai](#)[Tool Calling](#)

Written by:

piotr.minkowski

[View All Posts →](#)

2 COMMENTS



Geir March 14, 2025 7:00 am

Good article.

I will implement it in my agent.

<https://github.com/geir-eilertsen/marvin.robot/issues/94>

Loading...

[REPLY](#)



piotr.minkowski March 15, 2025 4:17 pm

Thanks!

Loading...

[REPLY](#)

Leave a Reply

Contact info

If you would like to contact me in order you have any questions, thoughts or ideas (e.g. suggestions for future articles) contact me via email.

 Warsaw, Poland
piotr.minkowski@gmail.com

Top Posts & Pages

-  [Claude Code Template for Spring Boot](#)
-  [Startup CPU Boost in Kubernetes with In-Place Pod Resize](#)
-  [Part 1: Creating microservice using Spring Cloud, Eureka and Zuul](#)
-  [Deep Dive into Kafka Offset Commit with Spring Boot](#)

Social Media

○ Speed up Java Startup with
Spring Boot and Project Leyden

Proudly powered by [WordPress](#) | Theme: [HoneyWaves](#) by [SpiceThemes](#)